

AI Medical Scribe

Expo mobile client & Express/MongoDB backend

System overview, external AI dependencies, and self-hosting requirements

Prepared: 3 September 2026

Status: Scaffold — written from the project brief; not yet verified against source

Audience: An engineer who has never seen this codebase

Purpose: Baseline for replacing hosted Sarvam models with a self-hosted LLM

Document status — read this first

This document has not been verified against the source code.

The two repositories named in the brief — the Expo mobile app and the Express/MongoDB backend — were not available when this document was generated. Nothing here was produced by reading the code.

Everything below comes from one of three places, and every claim is tagged so you can tell which:

- BRIEF** — asserted by the project owner in the specification for this document. Treated as true, but not independently confirmed.
- INFERENCE** — reasoned from the stated constraints. Plausible, explicitly not a fact about the code.
- VERIFIED** — read directly from source. **No claim in this revision carries this tag.**

Amber blocks mark the gaps: facts that can only come from the repositories. Do not circulate this document as authoritative to anyone building against this system until those blocks are filled and the tags upgraded.

Why the gaps were left as gaps

The brief asked for the file and function at each hop, the exact output shape a self-hosted replacement must emit, the terminology matcher's confidence thresholds, the MongoDB collections, and the full environment variable table. None of those are recoverable from a description of the system — only from the system. They have been left blank rather than filled with plausible-looking values.

This matters more than usual here. This is a clinical product, and this document's main purpose is to specify a replacement for the models that generate prescriptions. An invented field name in section 7 would produce a replacement service that silently fails to populate a prescription. An admitted gap costs an hour; a confident wrong sentence costs a patient.

How to complete it

Appendix A lists exactly what to supply and what each item unblocks.

Legend: **BRIEF** stated by owner **INFERENCE** reasoned, unconfirmed **VERIFIED** read from source

Contents

1. What the product is

- 1.1 Scope and shipped languages
- 1.2 Repository layout and deployment

2. End-to-end consultation flow

- 2.1 Pipeline diagram
- 2.2 Stage-by-stage walkthrough

3. External services

- 3.1 Sarvam Speech-to-Text
- 3.2 Sarvam Translate
- 3.3 Sarvam Chat
- 3.4 Resend
- 3.5 MongoDB Atlas

4. The report generation pipeline

- 4.1 Six section groups, two waves
- 4.2 The context mechanism
- 4.3 The SSE endpoint and early handover

5. Why the report is built this way

- 5.1 sarvam-105b is a reasoning model
- 5.2 The shared 4096-token budget
- 5.3 Consequences for schema size
- 5.4 What not to "clean up"

6. The local clinical terminology engine

- 6.1 Index and startup
- 6.2 Confidence thresholds and the drug-name rule

7. Replacing the hosted models

- 7.1 Two separate problems
- 7.2 Replacing Sarvam Chat
- 7.3 Replacing Sarvam Speech-to-Text
- 7.4 Replacing Sarvam Translate

8. Configuration reference

9. Known limitations and open issues

Appendix A — Intake checklist: what to supply to complete this document

1. What the product is

1.1 Scope and shipped languages

BRIEF The product is an AI medical scribe for doctors. During a consultation it records the doctor–patient conversation, transcribes it, and generates two artefacts: a structured clinical report and a printable prescription. The doctor reviews and edits the report on screen before exporting it.

The intent is to remove note-taking from the consultation itself. The doctor talks to the patient; the system produces the documentation. Because the output is a prescription, the doctor remains the author of record — the system drafts, the doctor approves.

BRIEF The application ships in English and Hindi.

GAP — NEEDS SOURCE

- Product name as it appears in the app and in `package.json`.
- Whether Hindi is UI localisation only, or whether reports and prescriptions are also generated in Hindi.
- The authentication and account model — who a "doctor" is in the data model, and whether there is a clinic/organisation tier above the individual user.
- Whether patient records persist between consultations or each consultation is standalone.
- Regulatory posture: any data-residency, retention, or consent handling present in the code.

1.2 Repository layout and deployment

Component	Stated stack	Deployment
Mobile client	React Native on Expo SDK 54, routing via <code>expo-router</code>	<i>Not stated</i>
Backend	Express with MongoDB, source under <code>server/</code>	Render

Both rows: **BRIEF**

GAP — NEEDS SOURCE

Directory tree of both repositories; language (TypeScript vs JavaScript) on each side; build and release process for the mobile app (EAS?); Render service configuration, including whether the backend runs as a single web service or has a separate worker for batch transcription polling.

2. End-to-end consultation flow

2.1 Pipeline diagram

BRIEF The stages below are the pipeline as described by the project owner. The boxes are correct; the file and function bindings under each box are the gaps.

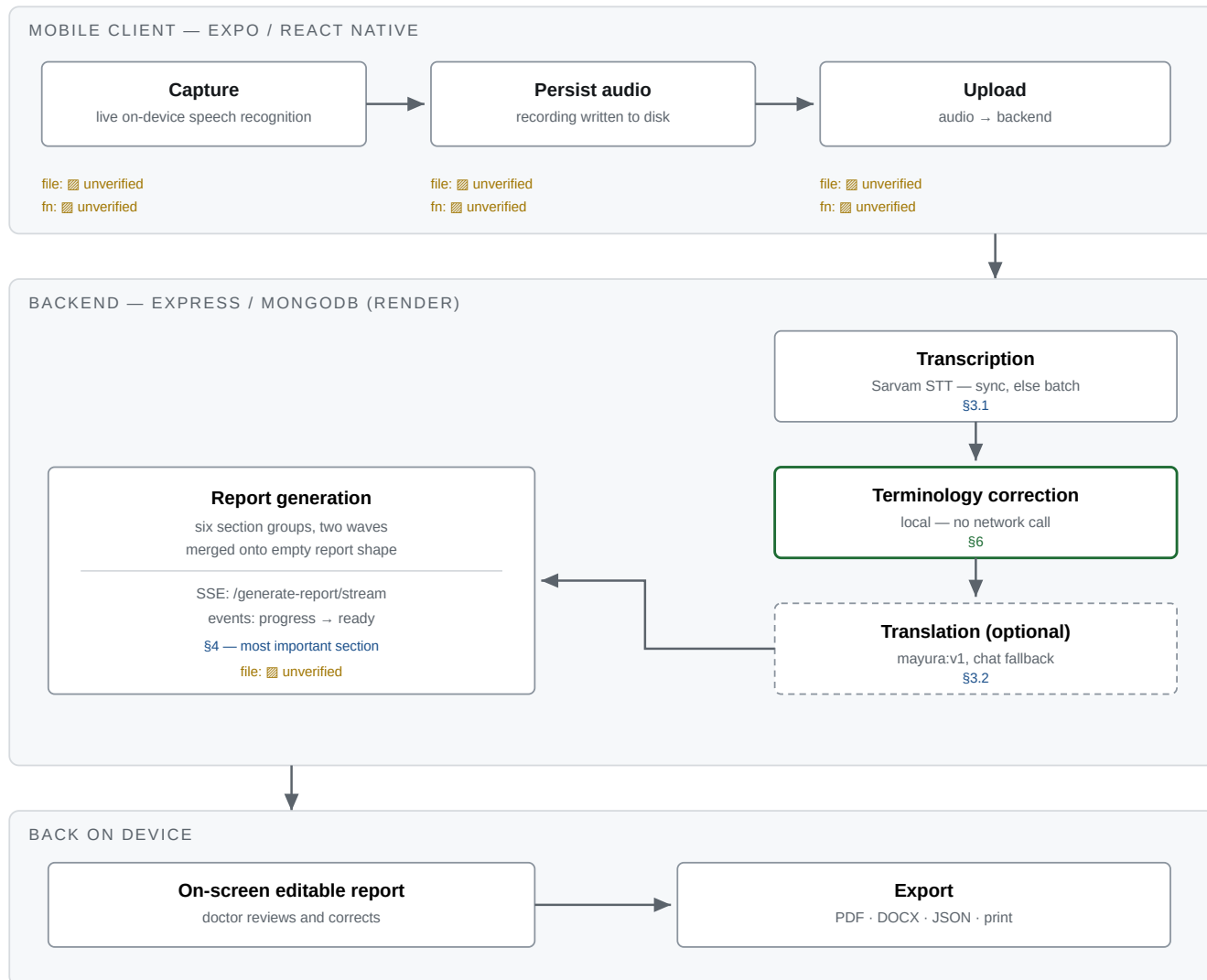


Figure 1 — Consultation pipeline. Solid border = always runs; dashed = conditional; green border = fully local. Amber "unverified" fields are the file/function bindings that require the repositories.

2.2 Stage-by-stage walkthrough

Capture

BRIEF Capture is two things happening at once: live on-device speech recognition, which gives the doctor immediate feedback that the consultation is being heard, and a persisted audio recording, which is what actually gets transcribed later. **INFERENCE** The on-device stream is therefore a UI affordance rather than the source of the clinical record — the authoritative transcript comes from the server-side Sarvam pass over the saved audio. This should be confirmed before anyone "optimises" by trying to reuse the on-device text.

Upload → transcription

BRIEF The saved audio is uploaded to the backend, which transcribes it via Sarvam. The backend attempts the synchronous endpoint first and falls back to the batch job flow. See §3.1 for why, and for the 30-second ceiling that forces the fallback in practice.

Terminology correction

BRIEF The raw transcript passes through a local clinical terminology engine that corrects misrecognised medical terms. No network call. See §6.

Optional translation

BRIEF Translation runs conditionally. See §3.2.

Report generation

BRIEF The corrected transcript is turned into a structured report through six independent extraction calls, not one. This is the subject of §4 and §5, and is the part of the system most likely to be broken by a well-meaning refactor.

Review and export

BRIEF The report is returned to the device, rendered editable, and exported as PDF, DOCX, JSON, or sent to a printer.

GAP — NEEDS SOURCE

For every hop above: the file, the exported function, the HTTP route and method, the request and response body shapes. Also:

- Which on-device speech recognition library is used, and what happens to its output — discarded, stored, or merged with the server transcript.
- Audio format, sample rate, and codec written to disk; whether it is re-encoded before upload.
- Upload transport: multipart to the backend, or direct-to-storage with a signed URL.
- The exact condition that triggers translation — user toggle, detected language, or target-language setting.
- Where PDF/DOCX rendering happens: on device or server-side.
- Failure and retry behaviour if the app loses connectivity mid-consultation, and whether audio is queued for later upload.

3. External services

Five external dependencies are named in the brief. Four are network services; one is a database. For each, latency figures and observed failure behaviour require either the code or production telemetry, and are marked as gaps throughout.

3.1 Sarvam Speech-to-Text

BRIEF Speech-to-text is consumed through two distinct surfaces, and the backend uses both.

Surface A — the real-time endpoint

Property	Value
Endpoint	/speech-to-text
Hard limit	30 seconds of audio
Model	saaras:v4
Role in the system	Attempted first for every transcription

INFERENCE A real consultation exceeds 30 seconds nearly always, so the synchronous attempt is expected to fail for most recordings and the batch path is the normal path, not the exceptional one. Anyone reading the code and seeing "sync first" should not assume sync is the common case. Confirm against production logs.

Surface B — the batch job flow

BRIEF The batch flow is a five-step sequence against `/speech-to-text/job/v1`:

1. **init** — create the job.
2. **upload-files** — obtain the upload target.
3. **PUT to Azure blob** — the audio is uploaded directly to Azure Blob Storage, not to Sarvam. Note this when planning a self-hosted replacement: there is an implicit second vendor in this path.
4. **start** — begin processing.
5. **poll** — repeatedly check for completion.

Model selection and the multi-speaker problem

BRIEF The model currently in use is `saaras:v4`. A variant, `saaras:v4-multispk`, exists and is purpose-built for multi-speaker audio — which is exactly what a doctor–patient consultation is. It is not usable, for two independent reasons:

- The account is rejected with the error *"currently in beta and not enabled for this subscription"*.
- It cannot auto-detect language, so it would require the language to be known up front.

INFERENCE The consequence is that the transcript arrives without reliable speaker attribution — the system cannot cleanly separate what the doctor said from what the patient said. Any downstream logic that appears to distinguish the two is doing so from linguistic cues in the report-generation prompts, not from diarisation. This is a significant clinical-accuracy consideration and should be confirmed in code.

Language codes

BRIEF A value of `unknown` in `language_code` means auto-detect.

GAP — NEEDS SOURCE

The full `language_code` mapping table — the application's internal language identifiers and the Sarvam code each maps to — is required and cannot be guessed. Sarvam's codes are locale-style strings, and writing an incorrect mapping into this document would be worse than leaving it empty.

GAP — NEEDS SOURCE

- Exact request bodies for all five batch steps, and the poll response shape including status enum values.
- Poll interval, timeout, and maximum attempts.
- Observed latency for sync and for a full batch cycle on a typical consultation.
- Cost per minute of audio under the current plan.
- What the user sees when transcription fails outright — retry, error state, or silent partial report.

3.2 Sarvam Translate

Property	Value
Endpoint	/translate
Model	mayura:v1
Fallback	Sarvam Chat (\$3.3)

All rows: [BRIEF](#)

Why a dedicated translation model rather than the chat model

INFERENCE The brief asks this to be explained, and the explanation follows from \$5 rather than from anything translation-specific. `sarvam-105b` is a reasoning model on a tier that caps `max_tokens` at 4096, shared between its reasoning trace and its answer. A translation request through the chat model therefore competes for that budget and can return an empty answer for a long transcript — the model spends the budget thinking and never emits the translation. `mayura:v1` is a purpose-built translation model with no reasoning trace, so its output budget is entirely output. It is also cheaper and lower-latency for a task that requires no reasoning.

The chat model remains as fallback, which is a reasonable design: if the dedicated model is unavailable, a degraded translation beats none.

GAP — NEEDS SOURCE

Request and response shapes; source/target language parameters; whether the transcript is chunked before translation and at what size; what triggers the fallback (HTTP status, timeout, empty result); whether the translated or original text is stored, or both.

3.3 Sarvam Chat

Property	Value
Endpoint	/v1/chat/completions
Model	sarvam-105b
Consumers	Report generation (\$4); translation fallback (\$3.2); in-app assistant

All rows: [BRIEF](#)

INFERENCE The endpoint path is OpenAI-compatible, which is materially good news for §7: a self-hosted replacement exposing an OpenAI-compatible API may need only a base-URL change rather than a rewrite of the calling code. This must be confirmed by reading the actual request construction — compatibility of the path does not guarantee compatibility of the fields used.

The behavioural constraints of this model are severe enough to have shaped the entire report architecture. They are documented separately in §5 and should be read before touching any code that calls this endpoint.

GAP — NEEDS SOURCE

Whether the client is the official OpenAI SDK pointed at Sarvam or a hand-rolled fetch wrapper; temperature and other sampling parameters; system prompts for each of the three consumers; retry policy; token accounting or cost tracking, if any. Also the in-app assistant's full scope — what it can see and whether it can modify the report.

3.4 Resend

BRIEF Resend is used for transactional email, and its only use is sending password-reset codes. It sits outside the clinical path entirely: if Resend is down, password resets stop and nothing else is affected.

GAP — NEEDS SOURCE

Sending domain and from-address; template content; code length, expiry, and rate limiting on reset attempts.

3.5 MongoDB Atlas

BRIEF MongoDB Atlas is the datastore.

GAP — NEEDS SOURCE

The brief asks for "collections and what each holds", and this cannot be inferred. Required: every collection name, its schema or Mongoose model, indexes, and — given this is patient data — where audio and transcripts are stored, how long they are retained, and whether anything is encrypted at rest beyond Atlas defaults.

4. The report generation pipeline

This is the section the brief marks as most important, and the one where the design is least likely to be guessed correctly by someone reading the code casually.

4.1 Six section groups, two waves

BRIEF The report is **not** produced by a single LLM call. It is extracted in six section groups:

Group	Covers	Wave
history	<i>Extraction scope unverified</i>	<i>unverified</i>
exam	<i>Extraction scope unverified</i>	<i>unverified</i>
plan	<i>Extraction scope unverified</i>	<i>unverified</i>
orders	<i>Extraction scope unverified</i>	<i>unverified</i>
systems	<i>Extraction scope unverified</i>	<i>unverified</i>
summary	<i>Extraction scope unverified</i>	<i>unverified</i>

BRIEF These six run in **two waves**. The results are then merged onto an empty report shape — a fixed skeleton of the full report structure, into which each group's output is placed. **INFERENCE** The empty-shape merge means a group that returns nothing degrades to blank fields rather than a failed report. Given §5, that is almost certainly deliberate: partial failure is expected and must not lose the groups that succeeded.

GAP — NEEDS SOURCE

Which groups are in wave one and which in wave two; the field list each group is responsible for; the empty report shape itself; and — critically — which group holds diagnosis and prescription (§5.3 states it is the smallest group in the file, but does not name it).

4.2 The context mechanism

BRIEF Later groups receive a `context` value showing what earlier groups already documented.

INFERENCE This exists because the six calls are otherwise blind to each other. Without it, two groups given the same transcript will both document the same finding, and the merged report will contradict or duplicate itself — a real clinical problem when the duplicate is a medication. The context is what makes six independent extractions read as one coherent note.

INFERENCE It also implies the two waves are ordered by dependency rather than convenience: wave one establishes the clinical facts, wave two writes with knowledge of them. A summary group that ran before the history group would have nothing to summarise.

GAP — NEEDS SOURCE

The exact shape of `context` — full prior JSON, a condensed digest, or prose; where it is injected (system prompt or user turn); and how its size is bounded, since it competes for the same 4096-token budget described in §5.

4.3 The SSE endpoint and early handover

BRIEF Report generation is exposed over Server-Sent Events at `/generate-report/stream`, emitting `progress` events and a `ready` event.

BRIEF The clinical core is handed to the doctor **before the last two groups finish**.

INFERENCE This is a deliberate latency trade, and it is the right one. Six sequential LLM calls against a reasoning model is a long wait with a patient in the room. The stages the doctor must actually verify — the clinical substance — are released as soon as they exist; the remaining groups arrive into an already-open report. The `ready` event is therefore not "generation complete", it is "enough of the report exists to be useful". A refactor that waits for all six before emitting `ready` would be functionally correct and a significant usability regression.

GAP — NEEDS SOURCE

- The full event set and each event's payload, including error events.
- Precisely which two groups complete after `ready`, and how the client merges late-arriving sections into a report the doctor may already be editing — this is a real conflict risk and the resolution strategy must be documented.
- Client-side reconnection behaviour if the SSE connection drops mid-generation.
- Whether a non-streaming report endpoint also exists.

5. Why the report is built this way

Read this before changing anything in §4.

The architecture in §4 looks over-engineered. Six calls where one would do, an odd two-wave split, a schema fragmented across groups, a hand-rolled context mechanism. Every one of those is a workaround for a hard platform constraint, and none of the constraints are visible in the code. An engineer who "simplifies" this will produce something that works in testing and returns empty prescriptions in production.

5.1 sarvam-105b is a reasoning model

BRIEF The model writes its reasoning into `reasoning_content` and leaves `content` **null** until it has finished reasoning.

Practical consequence: a response with `content: null` is not necessarily an error. It may be a model that ran out of budget mid-thought. Any calling code must handle a null `content` as a first-class outcome.

```
// Response shape to expect – content is null until reasoning completes
{
  "choices": [{
    "message": {
      "reasoning_content": "...the model's trace, which consumes budget...",
      "content": null           // ← not an exception; a normal outcome
    }
  }]
}
```

Illustrative shape based on the stated behaviour and the OpenAI-compatible envelope. Verify field names against an actual response before relying on them.

5.2 The shared 4096-token budget

BRIEF The subscription tier caps `max_tokens` at **4096**, and that budget is **shared between the reasoning trace and the answer**. Requesting 8192 is rejected outright with an explicit tier error.

So the usable output budget is not 4096. It is 4096 minus however much the model decided to think — a quantity that is not knowable in advance and varies with input complexity. A long, complicated consultation both needs a longer answer and provokes a longer reasoning trace. The two failure pressures compound.

5.3 Consequences for schema size

BRIEF A large requested schema regularly returns **nothing at all**. Not truncated output — nothing.

This is the direct cause of the §4 architecture. Six small extractions each fit inside the residual budget; one large extraction does not. The two-wave split, the fragmented schema, and the context mechanism all exist to keep each individual request small enough to survive.

BRIEF It is also why **diagnosis and prescription are isolated in the smallest group in the file**. These are the two fields whose absence is most dangerous, so they are given the greatest headroom against budget exhaustion. If someone merges that group into a larger one to reduce call count, the fields most likely to silently vanish are the diagnosis and the prescription.

The suppression flag does not work

BRIEF chat_template_kwargs.enable_thinking: false is **accepted** by the API but does **not reliably suppress** the reasoning trace.

```
// Accepted by the API. Does NOT reliably prevent the trace.
// Do not remove the small-group architecture on the strength of this flag.
{
  "model": "sarvam-105b",
  "max_tokens": 4096,                                // 8192 → explicit tier error
  "chat_template_kwargs": { "enable_thinking": false }
}
```

This is the most dangerous trap in the codebase. The flag is present, the API accepts it, and it looks like the reasoning problem is solved — so the workarounds around it look redundant. They are not.

5.4 What not to "clean up"

Tempting change	What actually happens
Merge the six groups into one call	Schema exceeds the residual budget; the call returns nothing; the report is empty
Move diagnosis/prescription into a bigger group	The two most safety-critical fields become the most likely to be dropped
Raise max_tokens to 8192	Rejected with a tier error; the request fails outright
Trust enable_thinking: false and drop the small-group design	The trace still appears, budget is still consumed, failures return intermittently
Treat content: null as an exception	Normal budget-exhaustion outcomes surface as crashes instead of degraded sections
Wait for all six groups before emitting ready	Doctor waits through the full chain with a patient present
Route translation through the chat model	Long transcripts return empty (§3.2)

Rows derive from the constraints stated in the brief; the consequences are reasoned, not observed. **INFERENCE**

6. The local clinical terminology engine

BRIEF Location: `server/services/terminology`. This component needs no API and makes no network call.

6.1 Index and startup

BRIEF It indexes roughly **485,000 terms** from `.tsv.gz` files committed to the repository, building the index at process startup, and corrects transcripts entirely in-process.

INFERENCE Two operational implications worth confirming: an index of that size has a memory footprint that must fit Render's instance limits, and building it at startup extends cold-start time — relevant if the service ever sleeps or scales from zero. Neither is a defect; both are things a new engineer should know before diagnosing a "slow first request".

GAP — NEEDS SOURCE

The `.tsv.gz` file list and what vocabulary each contains; the index data structure; measured startup time and resident memory; the matching algorithm (edit distance, phonetic, n-gram, or a combination); how the source data is refreshed.

6.2 Confidence thresholds and the drug-name rule

BRIEF The engine applies confidence thresholds to decide whether a candidate correction is applied. Fuzzy matches against **DRUG** names require a **much higher score** than other term categories.

The reasoning is stated plainly in the brief and deserves repeating in full, because it is the single most important safety rule in this component: **confusing two medicines is the worst failure this engine can produce.**

A wrongly "corrected" symptom is an annoyance a doctor will spot while reviewing. A wrongly corrected drug name is a plausible-looking, confidently-spelled medication that is not the one the doctor said. The asymmetry in thresholds is not tuning — it is a safety decision, and it should never be flattened for consistency.

GAP — NEEDS SOURCE

The actual numbers. Required: the default confidence threshold, the elevated DRUG threshold, every other category and its threshold, and what happens to a candidate that falls below — silently dropped, flagged to the doctor, or left as transcribed. These values are the operative content of this section and are deliberately not estimated here.

7. Replacing the hosted models

This is the reason the document exists. The section is structured as requirements on a replacement rather than as a recommendation of any particular model.

7.1 Two separate problems

The brief is right to insist these be treated separately. They are not the same engineering problem and they do not have the same difficulty.

Dependency	Nature of the problem	Portability
Report extraction (§4)	Prompt engineering against a structured-output task	Portable to any capable instruction-following model
In-app assistant	Prompt engineering	Portable
Translation (§3.2)	Indic translation quality	Moderate — needs Indic-capable model
Speech-to-text (§3.1)	Speech recognition for Indian languages and code-mixed Hindi-English	Hard. Not a prompting problem

BRIEF for the portability distinction; **INFERENCE** for the ordering.

The practical consequence: the chat model can likely be replaced first and independently, while speech-to-text may need to remain hosted for longer. These should be planned as separate migrations, not one cutover.

7.2 Replacing Sarvam Chat

A replacement must satisfy the following. Items marked as gaps cannot be specified without the code and are the blocking items for this migration.

Interface

INFERENCE The current endpoint is OpenAI-compatible (`/v1/chat/completions`). If the calling code uses only standard fields, any OpenAI-compatible server — vLLM, TGI, llama.cpp, Ollama — is a drop-in at the transport level. The exception is `chat_template_kwargs`, which is a Sarvam-specific extension; a replacement will ignore it, which is harmless given §5.3 says it does not work anyway.

Behavioural requirements

- **Must emit strictly parseable structured output** for each of the six section groups, matching the exact field names the merge step expects. A self-hosted model that produces correct clinical content in the wrong shape produces an empty report.
- **Should not require the small-group architecture** — a non-reasoning model with a real 4096+ output budget removes the §5 constraints entirely. But the architecture should be kept initially and simplified only after the new model is proven, so that a migration failure and an architecture failure cannot be confused.
- **Must handle the context mechanism** (§4.2) — the replacement receives prior-group output and must not re-document it.
- **Must tolerate the existing failure handling** — the calling code already handles null/empty content. A replacement that never returns null is strictly better; one that returns malformed JSON instead is worse, because that path may be less well covered.

GAP — NEEDS SOURCE · BLOCKING

The migration cannot be specified without: the six system prompts verbatim; the exact JSON schema each group is asked to emit; the empty report shape; the parsing and validation code, including what it does with malformed JSON; and the assistant's prompt and tool surface. This is the single highest-value item in Appendix A.

7.3 Replacing Sarvam Speech-to-Text

BRIEF Speech recognition needs Indian language support and code-mixed Hindi-English support. This is the genuinely hard requirement and the reason this dependency is listed last.

A doctor in an Indian consultation does not speak one language. A sentence commonly carries Hindi grammar with English clinical vocabulary — drug names, anatomy, and test names in English inside a Hindi utterance. A model strong on Hindi and a model strong on English can both fail this; the requirement is specifically code-mixed handling.

Requirements on a replacement

- **Input:** the persisted consultation audio. Format and duration gap-marked in §2.
- **Output:** a transcript in whatever shape the terminology engine (§6) consumes. **INFERENCE** Since the pipeline currently has no reliable diarisation (§3.1), a replacement that *does* provide speaker labels is an improvement but must not break a consumer expecting flat text.
- **Language handling:** must support auto-detection, since the current system relies on `unknown` meaning auto-detect and the language is evidently not always known in advance.
- **Duration:** must handle full consultations. The 30-second ceiling is a hosted-API constraint; a self-hosted model removes it and removes the entire batch/Azure-blob path with it — a substantial simplification and a second vendor eliminated.

INFERENCE Self-hosting ASR is where the largest architectural simplification is available, and also where clinical accuracy risk is highest. It should be evaluated on real consultation audio with drug names, not on benchmark datasets.

7.4 Replacing Sarvam Translate

Lowest risk of the three: translation is optional in the pipeline, already has a fallback path, and its failure mode is a missing translation rather than a wrong prescription. **INFERENCE** If the replacement chat model is Indic-capable, this dependency may simply collapse into it — provided the replacement is not a reasoning model with a shared budget, which was the original reason for separating them (§3.2).

8. Configuration reference

The brief asks for every environment variable the backend reads, what it does, and its default.

GAP — NEEDS SOURCE · THIS ENTIRE SECTION

Environment variable names cannot be inferred. A variable documented under the wrong name is worse than an undocumented one: it produces a service that starts, connects to nothing, and fails at the first consultation.

From the services described, the backend must read *at minimum* credentials for Sarvam, a MongoDB connection string, a Resend API key, and a port. Their actual names, defaults, and whether they are validated at boot are unknown.

Supply `.env.example`, the Render environment configuration, and a `grep` of `process.env` across `server/` to complete this table.

Variable	Required	Default	Purpose
Awaiting <code>.env.example</code> and <code>process.env</code> audit — intentionally left empty rather than populated with guessed names.			

9. Known limitations and open issues

Stated plainly, as requested. Items 1–6 derive from constraints in the brief; item 7 records what could not be assessed.

1. No usable speaker separation

BRIEF + **INFERENCE** The multi-speaker model is blocked at the subscription level and cannot auto-detect language. The system therefore transcribes a two-person conversation as undifferentiated text. For a product whose output is a clinical record, attributing a patient's statement to the doctor — or vice versa — is a meaningful accuracy risk. This is a vendor limitation, not a code defect, and it resolves either by getting the beta enabled or by self-hosting ASR (§7.3).

2. The synchronous transcription path is mostly decorative

INFERENCE A 30-second ceiling against consultation-length audio means the sync attempt fails routinely and the batch path carries the real traffic. Worth measuring: if sync succeeds effectively never, the attempt is pure added latency on every request.

3. Report generation can silently produce empty sections

BRIEF A group whose request exceeds the residual token budget returns nothing, and the merge onto an empty shape turns that into a blank section rather than an error. The mitigation — isolating diagnosis and prescription in the smallest group — reduces but does not eliminate the risk.

GAP — NEEDS SOURCE

Whether the doctor is told a section failed to generate, or whether a blank section is indistinguishable from "nothing to document". This distinction is clinically significant and should be verified as a priority.

4. An ineffective flag remains in the request payload

BRIEF `enable_thinking: false` is sent, accepted, and does not reliably work. It should carry an inline comment in the code; if it does not, it is a trap for the next engineer (§5.3).

5. Third-party storage in the transcription path

BRIEF The batch flow PUTs consultation audio to Azure Blob Storage. Patient audio therefore transits a storage vendor that is not otherwise part of the stack.

GAP — NEEDS SOURCE

Retention on that blob, whether it is deleted after transcription, and whether this is covered by the data handling the product claims to users.

6. Late-arriving report sections meet a report already being edited

INFERENCE The early-handover design (§4.3) means two groups land after the doctor has the report open. If the doctor is editing when they arrive, there is a write conflict. The code may handle this well; the risk is inherent to the design and is worth verifying explicitly.

7. Not assessed

The brief asks for "anything you find that looks unfinished". Nothing can be reported here — dead code, TODOs, unreachable branches, half-migrated modules, and failing tests are all findings that require reading the repositories. This heading is left open deliberately rather than closed with "none found", which would be false.

Appendix A — Intake checklist

What to supply, in priority order, and what each item unblocks.

#	Item	Unblocks
1	The report generation module — six group definitions, system prompts, JSON schemas, empty report shape, merge and parse code	§4 in full, §5 verification, §7.2 — the highest-value item
2	<code>server/services/terminology</code> source (indexer and matcher; not the <code>.tsv.gz</code> payloads)	§6 thresholds and the DRUG rule
3	The Sarvam client modules — STT sync and batch, translate, chat	§3 request/response shapes, language map, failure handling
4	<code>.env.example</code> plus a grep of <code>process.env</code> across <code>server/</code>	§8 entirely
5	Express route definitions and Mongoose models	§2 route bindings, §3.5 collections
6	Mobile app: recording, upload, SSE consumer, export screens	§2 client-side file/function bindings, §4.3 merge behaviour
7	One real Sarvam chat response including <code>reasoning_content</code> , and the verbatim 8192 tier error	Upgrades §5 samples from illustrative to verified

Delivery

Either a zip of both repositories — excluding `node_modules`, `.git`, `.expo`, build output, and the `.tsv.gz` data blobs — or public repository URLs for direct clone. Access tokens should not be shared.

On completion

Every **BRIEF** tag should be re-checked against the code and upgraded to **VERIFIED** or corrected. Every **INFERENCE** tag is a hypothesis to confirm or delete. The status banner on page 2 should be removed only when no amber blocks remain.